

# Distributed Systems — Individual Project

Docker & Local Kubernetes Orchestration

---

Adrien Dassie

MSc Data Engineering — Cohort MSc DE1

September 2026

GitHub repository:

`dataengineer-cloudcomputing/msc-de1-distributed-systems-docker-k8s`

Docker Hub image: `whale92400/msc-de1-flask-app`

## 1. Baseline: Proof the Original App Worked Before Docker

The starter application (UBC Flask Sample App) was forked and cloned locally. A Python virtual environment was created, dependencies were installed from `requirements.txt` (Flask 2.3.3), and the application was launched with `python run.py`, listening on `http://127.0.0.1:5000`.

One local environment issue was identified and documented before proceeding: macOS's built-in **AirPlay Receiver** service also listens on port 5000, and answered curl requests with a 403 response instead of the Flask app when a request resolved to IPv6. Forcing IPv4 (`curl -4`) resolved the ambiguity without disabling any system feature.

### All four documented routes responded correctly:

```
GET / -> "Hello, Flask!" (200)
GET /items -> {"items": []} (200)
GET /items/1 -> {"error": "Item not found"} (404)
POST /items {"name": "test"} -> {"message": "Item added successfully"} (201)
```

### Unit tests:

```
python -m unittest discover tests
Ran 4 tests in 0.111s
OK
```

Evidence (raw command output) is stored in `evidence/baseline.txt` in the repository.

## 2. Dockerization: Key Dockerfile Choices and Image Size

The application was not redesigned. The only functional change was replacing Flask's built-in development server with **Gunicorn** as a production-grade WSGI server, run with 2 sync workers bound to `0.0.0.0:5000`.

### Multi-stage build

A builder stage (based on `python:3.14-slim`) creates an isolated virtual environment and installs dependencies into it. The final runtime stage, based on the same slim image, copies only that pre-built virtual environment and the application source code — no compiler, package manager cache, or build-time artifact is present in the shipped image.

### Mandatory practices implemented

- Small official base image: `python:3.14-slim` (matches the local dev Python version).
- Clear `WORKDIR /app`.
- `requirements.txt` copied and installed before application source, for layer-cache efficiency.
- `pip install --no-cache-dir` to avoid leaving package-manager cache in the image.
- Only `app/`, `run.py` and `healthcheck.py` are copied into the final image.
- Dedicated non-root user `appuser` (uid/gid 999) via `USER appuser`.
- Only port 5000 is EXPOSEd.
- CMD given in exec form (`[ "gunicorn", ... ]`), so Gunicorn runs as PID 1 and receives `SIGTERM` directly for graceful shutdown, rather than being wrapped in a shell.
- A Docker-level HEALTHCHECK implemented as a small pure-Python script using `urllib` (already in the base image), avoiding the need to install `curl` and its extra attack surface.
- `PYTHONDONTWRITEBYTECODE=1` and `PYTHONUNBUFFERED=1` for clean, immediate container logs and to avoid runtime writes to the filesystem.

## Image inspection evidence

```
Final image content size : 55.5 MB
Container user identity : uid=999(appuser) gid=999(appgroup)
Exposed port : 5000/tcp only
Health status (docker ps): healthy
```

A `.dockerignore` file excludes `.git/`, the virtual environment, Python caches, IDE files, tests, and `report/evidence` folders from the build context.

## 3. Security: Non-root, Scan Findings, and SBOM

### Non-root execution

Confirmed identically across all three execution contexts: Docker (`docker exec ... id`), Docker Compose, and Kubernetes (pod security context) — all report `uid=999(appuser)` `gid=999(appgroup)`.

### Vulnerability scan

**Tool used:** Docker Scout CLI v1.21.0. **Image scanned:** flask-sample-app:1.0.0 (linux/arm64).

```
Before hardening : 57 vulnerabilities in 17 packages
After hardening : 53 vulnerabilities in 15 packages
CRITICAL 5
HIGH 10
MEDIUM 6
LOW 29
UNSPECIFIED 3
```

**What was changed after the scan:** the initial scan flagged 2 HIGH findings on `msgpack 1.1.2` and 1 HIGH + 1 MEDIUM on `setuptools 70.3.0`. Investigation with `pip show` and `find` inside the image showed that `msgpack` is not an application dependency at all — it is a copy vendored inside `pip` itself (`pip/_vendor/msgpack`), present both in the `venv` and in the base image's system Python (installed via `ensurepip`). Since Gunicorn serves the application directly from the virtual environment and never invokes `pip` at runtime, `pip`, `setuptools`, `pkg_resources` and `wheel` were removed from both locations in the final Dockerfile stage. This eliminated all four findings without touching any runtime dependency.

**Remaining findings:** the 53 remaining vulnerabilities (5 CRITICAL, 10 HIGH, ...) are all in Debian system packages shipped inside the official `python:3.14-slim` base image (`perl`, `glibc`, `pcr2`, `sqlite3`, `systemd`, `tar`, `gzip`, `coreutils`, and others). None of these binaries are ever executed by the Gunicorn/Flask application at runtime, so the practical exploitability is very low in this context. Removing them would mean patching the base OS image itself, which is outside the scope of an application-level Dockerfile and would risk breaking the base image's own integrity. This residual risk is accepted and documented rather than silently ignored, per the assignment's stated grading principle.

### Software Bill of Materials

Generated with `docker scout sbom --format spdx`, indexing 132 packages after hardening. Stored as `security/sbom.spdx.json` (SPDX format).

### Additional runtime hardening (Compose & Kubernetes)

- `read_only / readOnlyRootFilesystem`: `true` root filesystem, with a writable `tmpfs/emptyDir` mounted only at `/tmp`. Possible because the app keeps all data in memory and never writes `.pyc` files.

- `cap_drop: [ALL]` — no Linux capability is required (the app binds to an unprivileged port).
- `no-new-privileges / allowPrivilegeEscalation: false`.
- `seccompProfile: RuntimeDefault` in Kubernetes.

## 4. Docker Hub: Repository and Published Tags

Public repository: `whale92400/msc-de1-flask-app`  
(<https://hub.docker.com/r/whale92400/msc-de1-flask-app>).

- Tags published: `1.0.0` (initial version), `1.1.0` (adds the `/version` route for the rolling-update demonstration), and `latest`.
- After pushing, the local image cache was deliberately removed (`docker rmi`) and the image was re-pulled from Docker Hub to confirm the published artifact works standalone: the pulled container reached healthy status and answered `curl` correctly.
- Final Kubernetes deployment uses `whale92400/msc-de1-flask-app:1.0.0` after the rollback demonstration (see Section 6).

## 5. Kubernetes: Cluster Architecture and Deployed Objects

### Cluster topology (kind)

A local cluster named `msc-de1-cluster` was created with `kind`, using `kind/kind-config.yaml`: 1 control-plane node and 2 worker nodes, all reaching Ready status.

### Deployed objects (namespace `msc-de1-project`)

- **Namespace** — dedicated `msc-de1-project` namespace isolating all project resources.
- **Deployment** — 2 replicas, RollingUpdate strategy (`maxUnavailable=1`, `maxSurge=1`), readiness and liveness HTTP probes on `GET /`, CPU/memory requests (100m/64Mi) and limits (250m/128Mi), and a hardened pod/container security context (`runAsNonRoot`, `runAsUser: 999`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `readOnlyRootFilesystem: true`, `seccompProfile: RuntimeDefault`).
- **Service** — ClusterIP service exposing port 80, routed to container port 5000 via label selector.
- **NetworkPolicy** — restricts ingress to pods labelled `app: flask-app` to TCP port 5000 only.
- **ConfigMap** — non-sensitive environment configuration (`FLASK_ENV=production`).

**Known limitation:** `kind`'s default CNI (`kindnet`) does not enforce NetworkPolicy resources out of the box. The policy correctly documents the intended network isolation and would be enforced automatically if a policy-capable CNI such as Calico or Cilium were installed; this was not done here to keep the local cluster setup within the scope of the assignment.

### Verification performed

Both pods reached 1/1 Running, scheduled on the two different worker nodes. `kubectl get endpoints` confirmed the Service routes to both pod IPs. `kubectl exec ... id` confirmed non-root execution (`uid=999`) inside the cluster, matching Docker and Compose. The application was successfully reached end-to-end through the Service abstraction via `kubectl port-forward`.

## 6. Distributed Behavior: Self-healing, Scaling, Rolling Update and Rollback

### A. Self-healing

A running pod was deleted with `kubectl delete pod ...` while watching `kubectl get pods -w`. The deleted pod transitioned to `Terminating`; within seconds, a brand-new pod (`flask-app-755c6bdf8-xfdz8`) was created automatically by the `ReplicaSet` and reached `Running`, restoring the desired count of 2 replicas with no manual intervention. This demonstrates Kubernetes' continuous reconciliation between observed and desired state: the `ReplicaSet` controller watches the actual replica count and creates or removes pods whenever it diverges from the `spec.replicas` value.

## B. Scaling

`kubectl scale deployment flask-app --replicas=3` triggered the creation of a third pod (`flask-app-755c6bdf8-cxmcv`), confirmed at 3/3 ready. Scaling back down to `--replicas=2` gracefully terminated the excess pod, returning to a stable 2/2 state — the same reconciliation mechanism as self-healing, here triggered by an intentional specification change rather than a failure.

## C. Rolling update

A new route `GET /version` (returning `{"version": "1.1.0"}`) was added without touching any existing route, verified against the existing unit test suite (4/4 still passing), built and published as `whale92400/msc-de1-flask-app:1.1.0`. Updating the Deployment's image and applying it triggered a rolling update: `kubectl rollout status` confirmed "successfully rolled out", and both pods were verified to be serving image 1.1.0 (via `kubectl get pods -o jsonpath` and a live `curl /version` through the Service). `kubectl rollout history` recorded two revisions.

## D. Rollback

`kubectl rollout undo deployment/flask-app` reverted the Deployment to its previous revision. `kubectl rollout status` confirmed a successful rollout, and both pods were verified running the original `whale92400/msc-de1-flask-app:1.0.0` image again. This confirms that a Deployment retains enough revision history (previous `ReplicaSet` pod templates) to reverse an update on demand, without needing to manually rebuild or re-specify the prior state.

Full command output and explanations for all four demonstrations are stored under `evidence/k8s/` in the repository.

# 7. Conclusion

## What was learned

This project made the difference between a plain container and a Kubernetes-managed application concrete rather than theoretical. Building the Docker image forced explicit decisions about what a runtime image actually needs (a multi-stage build cut the image down to only the pre-built `virtualenv` and source code, and removing `pip/setuptools` after the fact — because they are build-time-only tools — measurably reduced the vulnerability surface reported by Docker Scout). Running the same image through Docker, Compose, and Kubernetes with a consistent non-root, capability-dropped, read-only-filesystem security posture showed how the same hardening choices carry across orchestration layers. Finally, the self-healing, scaling, and rolling-update/rollback demonstrations made visible what a Deployment's `ReplicaSet` actually does behind the declarative `spec.replicas` and `spec.template` fields: continuous reconciliation between desired and observed state, whether that gap is caused by a manual deletion, an intentional scale command, or a new image version.

## One improvement for a production deployment

The most significant gap between this local setup and a production deployment is network policy enforcement: the NetworkPolicy manifest expresses the intended ingress restriction, but kind's default kindnet CNI does not actually enforce it. In production, this project would need a policy-capable CNI (Calico or Cilium) so that the declared network isolation is genuinely applied, plus a proper Ingress controller with TLS termination in front of the Service, rather than relying on kubectl port-forward for access.